PRAXIS ENGINEERING PLATFORM • INDUSTRY READINESS SERIES

Software Developer Foundations Handbook

Full-Stack & Backend Systems Architecture, API Design, Git Protocols & Clean Code Standards

A definitive field reference for junior and transitioning software engineers navigating modern web services, distributed backend systems, clean application architectures, rigorous testing, and collaborative git workflows.

Curriculum Track:	Software Developer
Audience:	Junior Engineers, Transitioning Developers, Technical Interns
Platform Version:	Praxis Workplace Simulation v2.4 (Enterprise Edition)
Core Assessment:	5 MEQ Knowledge Check & 40 Production Engineering Problems
Publication Date:	October 2026 Edition • Confidential & Proprietary

NOTICE TO LEARNERS: This handbook serves as the foundational text for the Praxis Workplace Simulation. Thorough review of all 10 chapters is required prior to taking the 5-Question Readiness Assessment and accessing the 40 Production Problems (20 Basic with Help Statements + 20 Intermediate).

TABLE OF CONTENTS & CURRICULUM SYLLABUS

Overview of the 10 Core Knowledge Domains in this Handbook

Ch.	Chapter Title	Target Page
1	**Software Engineering Team Structure & Stakeholder Dynamics** How modern cross-functional product squads collaborate to deliver reliable user features.	Page 3 of 12
2	**The Modern Agile Cadence & Software Sprint Ceremonies** From sprint planning to retrospectives: mastering the two-week engineering heartbeat.	Page 4 of 12
3	**Git Hygiene, Conventional Commits & Code Review Etiquette** Trunk-based development, atomic commits, and creating high-empathy pull requests.	Page 5 of 12
4	**Software Architecture Tiers, Microservices & Data Modeling** Designing scalable, decoupled, and maintainable application architectures.	Page 6 of 12
5	**Automated Testing Pyramid, CI/CD Gates & Quality Assurance** Building trust through unit, integration, and contract test automation.	Page 7 of 12
6	**Application Telemetry: Logging, APM, Tracing & Profiling** Instruments of truth: observing real-time production software performance.	Page 8 of 12
7	**Production Debugging, Concurrency & Incident Triage** How elite engineers diagnose race conditions, deadlocks, and cascading outages.	Page 9 of 12
8	**Application Security Baseline: OWASP Top 10 & Secret Hygiene** Writing bulletproof software resistant to injection, auth bypass, and data leaks.	Page 10 of 12
9	**Software Engineering Lexicon: 40 Essential Industry Terms** The precise vocabulary required to communicate fluently in production engineering teams.	Page 11 of 12
10	**The First 90 Days Survival Guide for Junior Software Developers** Your week-by-week playbook for onboarding, building credibility, and shipping impact.	Page 12 of 12

Learning Invariant: *The 5-question knowledge check draws questions directly from these 10 chapters. Each syllabus topic corresponds to practical scenarios in the 40 production problems.*

CHAPTER 1: SOFTWARE ENGINEERING TEAM STRUCTURE & STAKEHOLDER DYNAMICS

How modern cross-functional product squads collaborate to deliver reliable user features.

In modern software organizations, engineers operate inside cross-functional squads comprising Product Managers (PMs), Product Designers (UX/UI), QA Engineers, and fellow developers. The PM governs the 'why' and 'what' (prioritized customer problem statements), whereas the engineering team exclusively governs the 'how' (technical architecture, system design, scalability, and code quality). Technical decisions must balance rapid delivery against technical debt accrual.

FIELD STANDARD: GOLDEN COMMUNICATION STANDARD

Never state 'the feature is almost done' without defining concrete criteria. Specify 'PR open and reviewed; awaiting integration test green status and stage smoke testing.'

Role	Primary Responsibility	Critical Interface with Developers
Product Manager (PM)	Product strategy, user requirements & PRDs	Feature scope negotiation, acceptance criteria
Staff / Tech Lead	System architecture & technical standard setting	Design reviews, non-functional requirement validation
Frontend Engineer	Client rendering, state machines & Web Vitals	API contract agreements, mock payloads, error states
Backend Engineer	API routes, business logic & persistence tier	Database indexing, schema migrations, concurrency control
QA Engineer	Automated regression testing & test matrix	Edge-case analysis, synthetic test fixture alignment

CHAPTER 2: THE MODERN AGILE CADENCE & SOFTWARE SPRINT CEREMONIES

From sprint planning to retrospectives: mastering the two-week engineering heartbeat.

Agile is not bureaucratic overhead; it is a rapid feedback loop designed to mitigate uncertainty. Teams execute in 2-week iterations (sprints). The sprint backlog represents a committed forecast. Work is decomposed into User Stories accompanied by verifiable Acceptance Criteria (Gherkin style: Given-When-Then). Engineers are expected to raise blockers immediately rather than waiting for ceremonies.

FIELD STANDARD: THE 3 STANDUP QUESTIONS VS IMPACT FRAMING

Instead of saying 'I worked on tickets yesterday', frame as: 'Completed auth token refresh logic; today pairing on the idempotency middleware. No blockers, on track for sprint demo.'

Ceremony	Frequency & Duration	Software Engineer's Deliverable
Sprint Planning	Every 2 weeks (90 mins)	Estimate user stories with Fibonacci story points; decompose into tasks
Daily Standup	Daily (15 mins)	Share progress towards sprint goal, flag blockers, identify pairing needs
Backlog Refinement	Weekly (45 mins)	Clarify acceptance criteria, flag technical spikes, identify edge cases
Sprint Review & Demo	End of Sprint (45 mins)	Demonstrate working code in staging environment to stakeholders
Retrospective	End of Sprint (45 mins)	Blameless analysis of what went well, what stalled, and action items

CHAPTER 3: GIT HYGIENE, CONVENTIONAL COMMITS & CODE REVIEW ETIQUETTE

Trunk-based development, atomic commits, and creating high-empathy pull requests.

Your Git commit history is the living ledger of your application's architecture. Elite engineering organizations employ Trunk-Based Development with short-lived feature branches (< 24 hours). Commits must be atomic, bisectable, and formatted according to the Conventional Commits specification (e.g. feat:, fix:, refactor:, test:, chore:). Pull Requests should remain small (< 400 lines) to ensure high review quality and rapid merge velocity.

FIELD STANDARD: PULL REQUEST ETIQUETTE RULE

Always self-review your PR before requesting peer reviews. Provide screenshots or video for UI changes, and include curl commands or test outputs for API changes.

Commit Prefix	Semantic Intent	Example Git Commit
feat(auth)	New user-facing functionality	feat(auth): implement rotating refresh token rotation on logout
fix(billing)	Bug remediation with zero feature creep	fix(billing): handle floating-point cent rounding in invoice tax
refactor(db)	Code structure change without behavioral drift	refactor(db): extract connection pool singleton into dependency injector
test(users)	Adding or correcting automated test suites	test(users): add parameterized edge cases for unicode email validation
perf(search)	Targeted latency or resource optimization	perf(search): add composite b-tree index on (org_id, created_at)

CHAPTER 4: SOFTWARE ARCHITECTURE TIERS, MICROSERVICES & DATA MODELING

Designing scalable, decoupled, and maintainable application architectures.

Modern web applications adhere to Clean Architecture principles: Separation of Concerns, Inversion of Control, and Explicit Domain Boundaries. Software is segmented into Presentation (API routes/controllers), Application (orchestration & use-cases), Domain (pure business entities and invariant rules), and Infrastructure (databases, message queues, third-party HTTP clients).

FIELD STANDARD: IDEMPOTENCY IN API SYSTEMS

Any state-changing POST/PATCH request must accept an 'Idempotency-Key' header. If a network blip occurs, the client retries safely without causing duplicate charges or data corruption.

Architectural Layer	Primary Responsibility	Forbidden Anti-Pattern
Controller / Presentation	Request deserialization, HTTP status codes, validation	Executing raw SQL queries or complex business logic inside route handlers
Domain Services	Pure core business rules, entity invariants, calculations	Importing database drivers or framework-specific HTTP contexts
Data Access (Repository)	Query optimization, connection pooling, transactions	Leaking database entities directly into external public API payloads
Caching Tier (Redis)	Sub-millisecond reads, distributed locks, session storage	Treating cache as permanent storage without cache invalidation strategies

CHAPTER 5: AUTOMATED TESTING PYRAMID, CI/CD GATES & QUALITY ASSURANCE

Building trust through unit, integration, and contract test automation.

Reliability cannot be tested into software at the end; it must be designed into the codebase from the initial commit. The Test Automation Pyramid mandates a broad base of fast, isolated Unit Tests (pure logic), a focused mid-tier of Integration Tests (database queries, repository contracts, Redis caching), and a narrow top tier of End-to-End (E2E) workflow tests running in ephemeral environments.

FIELD STANDARD: CONTINUOUS INTEGRATION (CI) INVARIANT

A broken CI build on 'main' halts all feature work. The engineer who merged the breaking commit is immediately responsible for fixing forward or reverting within 10 minutes.

Testing Tier	Execution Speed	Targeted Failure Mode	Mocking Strategy
Unit Tests	< 1 ms per test	Logic branches, math errors, edge-case null checks	Mocks all network and disk I/O
Integration Tests	10 - 200 ms per test	SQL syntax, migration locks, schema mismatches	Real Dockerized Postgres / Redis testcontainers
Contract Tests	50 - 500 ms per test	API schema drift between frontend and backend	Pact / OpenAPI schema validator
End-to-End (E2E)	5 - 30 s per test	Broken user checkout, state machine regression	Synthetic headless browser runs (Playwright)

CHAPTER 6: APPLICATION TELEMETRY: LOGGING, APM, TRACING & PROFILING

Instruments of truth: observing real-time production software performance.

Engineers cannot fix what they cannot observe. Modern production software must emit structured JSON logs with correlated trace identifiers (`trace_id`, `span_id`). Application Performance Monitoring (APM) instruments call stacks to pinpoint CPU hotspots, database query N+1 bottlenecks, and third-party API latency spikes. Unstructured log strings (e.g. `console.log('error here')`) are strictly prohibited.

FIELD STANDARD: STRUCTURED LOGGING IMPERATIVE

Always emit JSON logs with key metadata: {'timestamp': '2026-10-22T08:00:00Z', 'level': 'ERROR', 'trace_id': 'abc-123', 'user_id': 'u_456', 'error_code': 'DB_LOCK_TIMEOUT'}.

Telemetry Pillar	Key Tooling	Diagnostic Utility for Software Engineers
Structured Logs	Winston / Loguru / ELK	Contextual forensics: exact parameters and error traces during failures
APM & Distributed Traces	Datadog / OpenTelemetry / Jaeger	End-to-end request hop analysis: isolating slow DB queries and microservice latency
Application Metrics	Prometheus / StatsD	Time-series counters (requests/sec), gauges (pool active), histograms (p99 duration)
Crash Reporting	Sentry / Bugsnag	Real-time exception grouping, user session breadcrumbs, stack frame mapping

CHAPTER 7: PRODUCTION DEBUGGING, CONCURRENCY & INCIDENT TRIAGE

How elite engineers diagnose race conditions, deadlocks, and cascading outages.

When production incidents occur, engineers must transition into systematic triage mode: Mitigate First, Diagnose Offline. Common software failure modes include thread pool exhaustion, database connection deadlocks, unindexed table scans causing query queueing, and unbounded memory allocations triggering out-of-memory (OOM) process termination.

FIELD STANDARD: THE CIRCUIT BREAKER & BULKHEAD PATTERNS

When calling upstream services, always wrap client calls in circuit breakers. If an external API slows down, fail fast immediately to preserve internal thread pools and prevent cascading failure.

Failure Symptom	Underlying Root Cause	Immediate Engineering Remediation
Database Lock Timeout	Concurrent transactions updating rows in differing order	Enforce strict alphabetical lock ordering across all queries
High Latency on List API	N+1 query problem loading foreign keys in a loop	Replace loop with batch JOIN or eager loading with IN clause
Process OOM (Exit 137)	Reading entire 5GB file/query into memory at once	Refactor to streaming chunk reader / database cursors
504 Gateway Timeout	Synchronous third-party HTTP call in critical path	Decouple into background asynchronous queue (Celery/Kafka)

CHAPTER 8: APPLICATION SECURITY BASELINE: OWASP TOP 10 & SECRET HYGIENE

Writing bulletproof software resistant to injection, auth bypass, and data leaks.

Security is not a downstream QA checklist; it is an intrinsic software engineering discipline. Every incoming parameter is untrusted until sanitized and validated. Developers must protect against the OWASP Top 10 vulnerabilities: Broken Access Control, Cryptographic Failures, Injection (SQLi/XSS), and Insecure Design. Hardcoded secrets in codebases represent severe security incidents.

FIELD STANDARD: ZERO SECRETS IN GIT RULE

Never commit API keys, database passwords, or JWT signing secrets to git repositories. Always use environment variables injected via Vault or Kubernetes Secrets, backed by pre-commit hooks.

Vulnerability	Attack Vector	Mandatory Engineering Defense
SQL Injection (SQLi)	Tainted string interpolation in SQL queries	Parameterized queries, prepared statements, typed ORM models
Cross-Site Scripting (XSS)	Unsanitized user text injected into browser DOM	Context-aware HTML escaping, Content Security Policy (CSP) headers
Broken Object Level Auth (BOLA)	User accessing /api/invoices/999 without tenant check	Enforce row-level ownership validation on every database query
CSRF & Session Hijacking	Malicious site forging authenticated state changes	HttpOnly, Secure, SameSite=Strict cookie flags + CSRF tokens

CHAPTER 9: SOFTWARE ENGINEERING LEXICON: 40 ESSENTIAL INDUSTRY TERMS

The precise vocabulary required to communicate fluently in production engineering teams.

Fluency in industry vocabulary enables high-bandwidth, precise technical collaboration without ambiguity. Master these 40 terms to understand architecture RFCs, system design discussions, and production incident postmortems.

FIELD STANDARD: LEXICON INVARIANT

Use precise architectural terminology during design reviews. Distinguish clearly between 'Authentication' (who you are) and 'Authorization' (what you can do).

Term	Industry Definition & Practical Engineering Context
Idempotency	A property of an API operation where making multiple identical requests has the exact same side-effect as a single request.
Database Index	A b-tree data structure that improves the speed of data retrieval operations on a database table at the cost of slower writes.
Connection Pooling	A cache of database connections maintained so that connections can be reused rather than opened and closed for every query.
Race Condition	A bug where the system's output depends on the timing or sequence of uncontrollable events like concurrent thread execution.
Deadlock	A situation where two or more concurrent transactions are unable to proceed because each holds a lock the other needs.
N+1 Query Problem	A performance anti-pattern where an application executes 1 query to retrieve N records, and then N additional queries for child data.
Circuit Breaker	A software design pattern used to detect failures and encapsulate the logic of preventing a failure from constantly recurring.
Backpressure	A mechanism that allows a receiving system to signal to a sending system that it is overwhelmed and needs it to slow down.
Eventual Consistency	A consistency model used in distributed systems where all replicas will eventually converge to the same state.
Clean Architecture	A software design philosophy that isolates core business domain logic from frameworks, UI, and persistence tiers.

CHAPTER 10: THE FIRST 90 DAYS SURVIVAL GUIDE FOR JUNIOR SOFTWARE DEVELOPERS

Your week-by-week playbook for onboarding, building credibility, and shipping impact.

Your first 90 days establish your reputation as a reliable, curious, and rigorous engineer. Success is measured not by how fast you write code, but by how reliably you learn the domain, adhere to team standards, write comprehensive tests, and communicate proactively.

FIELD STANDARD: THE 30-MINUTE BLOCKED RULE

When stuck, spend 30 minutes researching and isolating the problem. If still blocked, ask your mentor: document what you tried, what you expected, what happened, and your working hypothesis.

Timeline Milestone	Key Objective & Deliverable	Success Metric
Week 1: Setup & Environment	Clone repo, run full test suite locally, execute first non-prod deploy	Environment working without mentor intervention; zero secrets leaked
Week 2: First Production Bug	Pick up a well-scoped bugfix ticket, write reproduction test, open PR	PR merged with clean commit message and passing CI pipeline
Day 30: Feature Contributor	Deliver first full user story including database migration and tests	Code shipped to production with zero regressions or rollbacks
Day 60: System Understanding	Document a tribal knowledge workflow; shadow an on-call engineer	Conducted architecture walkthrough of one core microservice
Day 90: Autonomous Developer	Own a full sprint feature from refinement to deployment and monitoring	Proactively proposed test improvements; trusted peer reviewer